ServiX

Especificación de requisitos de software

Estructura ISO/IEC/IEEE 29148:2018 con mapeo a IEEE 830-1998

Campo	Detalle
Proyecto	ServiX: aplicación Android para gestión de servicios técnicos
Asignatura	Optativa II · Gestión de proyectos integradores con Scrum
Equipo	Equipo ServiX (Juan Esteban Olaya Muñoz, Nicolas Quiroz Gomez, Juan Sebastian Garzon Suarez)
Versión / fecha	1.0 · 01/11/2026

Tabla de contenido

Tabla de contenido 2

1. Introducción 3

1.1 Propósito . . . 3

1.2 Alcance . . . 3

1.3 Normas de referencia . . . 3

1.4 Definiciones y abreviaturas . . . 3

1.5 Visión general del documento . . . 3

2. Descripción general 3

2.1 Perspectiva del producto . . . 4

2.2 Funciones del producto . . . 4

2.3 Usuarios . . . 4

2.4 Restricciones . . . 4

2.5 Supuestos y dependencias . . . 5

3. Requisitos específicos 5

3.1 Interfaces externas . . . 5

3.2 Requisitos funcionales . . . 5

3.3 Requisitos no funcionales . . . 7

3.4 Modelo de datos lógico . . . 7

3.5 Restricciones de diseño y seguridad . . . 8

4. Verificación y trazabilidad 8

4.1 Métodos de verificación . . . 8

4.2 Trazabilidad requisito a casos de prueba . . . 8

5. Mapeo a IEEE 830-1998 9

6. Limitaciones de este documento 9

1. Introducción

1.1 Propósito

Este documento especifica los requisitos de **ServiX**, una aplicación Android para gestionar servicios técnicos del hogar: clientes, órdenes de servicio, historial, notificaciones y un asistente de chat con IA que puede escalar casos a un administrador. Está dirigido al equipo de desarrollo, al Product Owner, al Scrum Master y al docente evaluador.

Los requisitos se derivan **de las funcionalidades que existen en el código fuente** (`app/src/main/java/com/example`) y no de funcionalidades planeadas.

1.2 Alcance

ServiX permite a un administrador o vendedor gestionar su cartera de clientes y sus servicios, y a un cliente solicitar servicios y atención desde un portal propio. Incluye autenticación con Firebase, persistencia en Cloud Firestore, un chat con Gemini mediante Firebase AI Logic y notificación de eventos a Make. Quedan fuera de alcance: pagos en línea, notificaciones push, llamadas con Retell AI y flujos con n8n (existen solo como interfaces sin implementación).

1.3 Normas de referencia

- ISO/IEC/IEEE 29148:2018, Systems and software engineering, Life cycle processes, Requirements engineering (estructura principal de este SRS).
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications (sustituida por 29148; se incluye el mapeo en la sección 5).
- IEEE Std 1016-2009, Software Design Descriptions (documento de arquitectura y diseño).
- ISO/IEC/IEEE 29119 (partes 2 y 3, 2021), Software testing (plan e informe de pruebas).

Aclaración sobre las normas. Este documento usa la estructura y las prácticas de las normas citadas como guía de organización. No constituye una certificación ni una declaración de conformidad con IEEE o ISO.

1.4 Definiciones y abreviaturas

Término	Definición
Administrador / Vendedor	Rol `ADMINISTRADOR` de la aplicación: gestiona clientes, servicios y solicitudes de atención.
Cliente	Rol `CLIENTE`: usa el portal del cliente y el chat.
PBI	Product Backlog Item de Azure Boards.
Escalamiento	Creación de una solicitud de atención humana desde el chat.
Firebase AI Logic	Servicio de Firebase para llamar a modelos Gemini sin exponer claves en la app.
Mock	Repositorio en memoria usado en pruebas.

1.5 Visión general del documento

La sección 2 describe el producto; la 3 contiene los requisitos específicos con criterios de aceptación; la 4 define la verificación y la trazabilidad; la 5 mapea la estructura a IEEE 830. Las matrices completas están en `02_Matrices/Matriz_Maestra.xlsx`.

2. Descripción general

2.1 Perspectiva del producto

ServiX es una aplicación de una sola actividad (`MainActivity`) construida con Jetpack Compose y Material 3. Usa los servicios de Firebase (Authentication, Cloud Firestore y AI Logic) y un webhook de Make. La integración continua se ejecuta en Azure Pipelines.

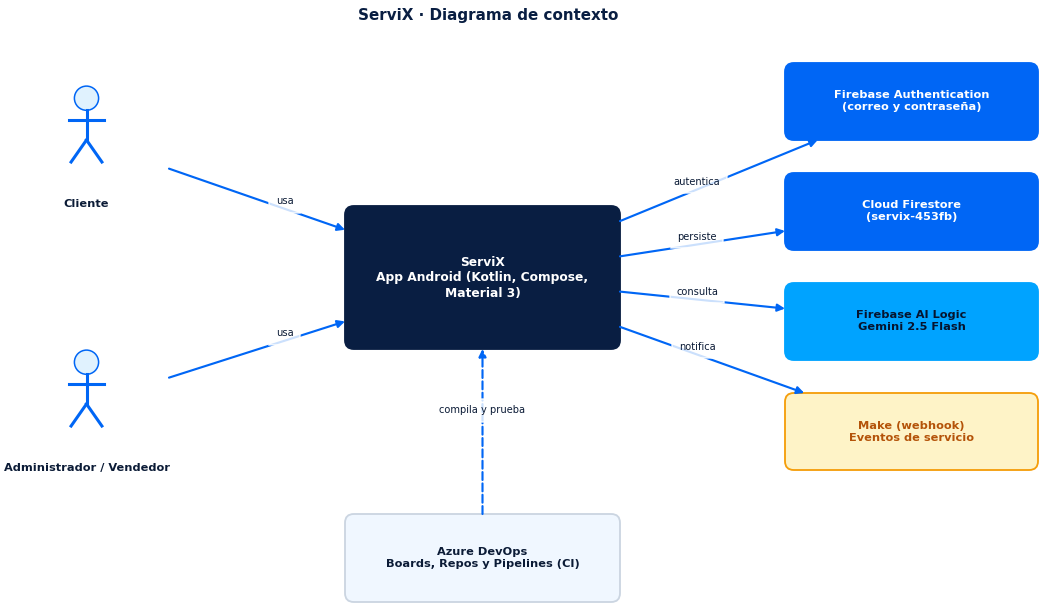


Figura 1. Diagrama de contexto de ServiX.

2.2 Funciones del producto

- Autenticación con dos roles (Cliente y Administrador) y recuperación de contraseña.
- Gestión de clientes y servicios con seguimiento de estados.
- Historial, notificaciones, búsqueda global y metas semanales.
- Asistente de chat con IA, clasificación de intención y escalamiento a una persona.
- Portal del cliente y gestión de solicitudes de atención.
- Notificación de eventos a Make.

2.3 Usuarios

Rol	Descripción	Pantallas principales
Cliente	Persona que solicita servicios o atención.	Portal del cliente (`cliente_portal`), Chat
Administrador / Vendedor	Gestiona el negocio.	Inicio, Clientes, Servicios, Historial, Chat IA, Perfil

2.4 Restricciones

- Plataforma Android 8.0 (API 26) o superior; `targetSdk` 36.
- Kotlin 2.2, Jetpack Compose, Material 3, Android Gradle Plugin 9.1.
- Backend gestionado por Firebase (proyecto `servix-453fb`); no hay servidor propio.

- Equipo de desarrollo con recursos limitados (4 GB de RAM): las compilaciones locales son lentas.

2.5 Supuestos y dependencias

- Firebase AI Logic está activado con Gemini Developer API (estado al 31/10/2026).
- El usuario dispone de conexión a Internet para autenticación, Firestore y chat.
- El webhook de Make está disponible para los eventos de servicio.

3. Requisitos específicos

3.1 Interfaces externas

Interfaz	Tipo	Descripción
Interfaz de usuario	Gráfica	Pantallas Compose con navegación inferior para el administrador y portal para el cliente.
Firebase Authentication	Servicio	Correo y contraseña; recuperación de contraseña.
Cloud Firestore	Servicio	Datos por usuario `usuarios/{uid}/…` y `solicitudes_atencion`.
Firebase AI Logic (Gemini)	Servicio	Modelo `gemini-2.5-flash`, temperatura 0,7, máximo 800 tokens de salida.
Webhook de Make	HTTP	Eventos `SERVICIO_CREADO` y `SERVICIO_COMPLETADO` enviados con OkHttp.

3.2 Requisitos funcionales

Cada requisito indica su prioridad, su criterio de aceptación y su estado de cumplimiento. El estado refleja la evidencia real a la fecha (01/11/2026).

Estados de cumplimiento. Cumplido: implementado y verificado con pruebas automáticas ejecutadas o evidencia concreta. Parcialmente cumplido: implementado, con verificación parcial o sin verificación en dispositivo o servicio real. No cumplido: no implementado. No verificable: no puede determinarse sin ejecutar la app, un dispositivo o un servicio externo.

ID	Requisito y descripción	Criterio de aceptación	Prior.	Estado
RF-001	**Registro de cuenta.** El usuario crea una cuenta con nombre, correo, contraseña, teléfono opcional y rol.	Se rechaza nombre vacío, correo sin '@' y contraseña de menos de 6 caracteres; con datos válidos se crea la cuenta y el perfil usuarios/{uid} con su rol.	Alta	Parcialmente cumplido
RF-002	**Inicio de sesión con validación de rol.** El usuario inicia sesión con correo y contraseña; el rol se toma de la cuenta almacenada y, si no coincide con el elegido, se rechaza el acceso.	Contraseña de menos de 6 caracteres devuelve 'La contraseña debe tener al menos 6 caracteres'; un rol distinto al almacenado rechaza el acceso.	Alta	Parcialmente cumplido
RF-003	**Cierre de sesión.** El usuario cierra sesión desde Perfil o Portal y vuelve a la pantalla de autenticación.	Tras cerrar sesión no existe usuario actual y la navegación vuelve a la ruta auth.	Alta	Cumplido
RF-004	**Recuperación de contraseña.** El usuario solicita el envío de un correo de restablecimiento.	Con correo vacío se muestra 'Ingresa tu correo para recuperar contraseña'; con correo válido se solicita el envío a Firebase.	Media	No verificable
RF-005	**Navegación según rol.** Tras autenticarse, el cliente ve el Portal del cliente y el administrador ve el panel con barra inferior (Inicio, Clientes, Servicios, Historial, Chat IA, Perfil).	Rol CLIENTE navega a cliente_portal; rol ADMINISTRADOR navega a home; sin sesión se muestra auth.	Alta	Parcialmente cumplido
RF-006	**Gestión de clientes: alta y edición.** El administrador registra y edita clientes (nombre, documento, teléfono, correo, dirección, notas, foto).	El cliente creado recibe identificador, aparece en la lista y conserva sus datos; las iniciales se calculan del nombre.	Alta	Cumplido

ID	Requisito y descripción	Criterio de aceptación	Prior.	Estado
RF-007	**Eliminación y restauración de clientes.** El administrador elimina un cliente (junto con sus servicios) y puede deshacer la operación.	Al eliminar un cliente se eliminan sus servicios asociados; restaurar recupera cliente y servicios.	Media	Cumplido
RF-008	**Favoritos.** El administrador marca clientes y servicios como favoritos.	El indicador de favorito cambia y se persiste.	Baja	Parcialmente cumplido
RF-009	**Segmentación automática de clientes.** Cada cliente recibe segmentos (VIP, Frecuente, Nuevo, Pendiente, Inactivo) según sus servicios y antigüedad.	VIP con facturación completada ≥ 500.000 o 3 servicios completados; Frecuente con 2 o más servicios; Pendiente con servicios abiertos; Nuevo con menos de 15 días y 1 servicio como máximo.	Media	Parcialmente cumplido
RF-010	**Gestión de servicios.** El administrador crea, edita, duplica y elimina servicios con cliente, categoría, precio, fechas, prioridad y observaciones.	El servicio nuevo queda en estado Pendiente y con prioridad Media por defecto; duplicar crea una copia; eliminar permite restaurar.	Alta	Cumplido
RF-011	**Seguimiento del estado del servicio.** El administrador cambia el estado de un servicio (Pendiente, En Proceso, Completado, Cancelado) y ve el progreso en un stepper.	El cambio de estado se refleja en la lista y en el detalle; al completar o cancelar se registra la actividad.	Alta	Cumplido
RF-012	**Historial de actividades y resumen financiero.** El sistema registra actividades (alta, edición, eliminación de clientes y servicios) y las presenta en Historial junto con el Centro Financiero.	Cada operación relevante genera una actividad con título, descripción y fecha.	Media	Parcialmente cumplido
RF-013	**Notificaciones internas.** El sistema genera notificaciones (servicio de hoy, urgente, pendiente, completado, nuevo cliente, seguimiento, solicitud de atención) y permite marcarlas como leídas o eliminarlas.	Una solicitud de atención genera una notificación para el administrador; marcar como leída actualiza el contador.	Media	Parcialmente cumplido
RF-014	**Búsqueda global.** El administrador busca en clientes, servicios, notas y categorías desde una sola ventana.	Al escribir se listan coincidencias; sin resultados se muestra un mensaje de ausencia de coincidencias.	Media	No verificable
RF-015	**Panel de inicio y metas semanales.** El administrador consulta estadísticas, servicios del día y define metas semanales de servicios e ingresos.	Las metas se guardan en la configuración (por defecto 10 servicios y 1.200.000 de ingresos).	Media	Parcialmente cumplido
RF-016	**Perfil del negocio y configuración.** El administrador edita el perfil del negocio, foto, modo oscuro, alertas, autoguardado y moneda.	Los cambios de perfil y configuración se guardan y persisten.	Media	Parcialmente cumplido
RF-017	**Tutorial de bienvenida.** La app muestra un tutorial inicial y recuerda que fue visto.	Tras marcarlo como visto no vuelve a mostrarse.	Baja	No verificable
RF-018	**Asistente de chat con IA.** El usuario conversa con un asistente que responde con Gemini usando como contexto sus servicios, clientes y perfil.	Con consultas generales se envía el contexto del usuario a Gemini (gemini-2.5-flash) y se muestra la respuesta; ante falla se muestra un mensaje de error sin simular una respuesta de IA.	Alta	No verificable
RF-019	**Clasificación de intención y escalamiento.** El chat clasifica la intención por palabras clave y, ante queja, cancelación, problema, error de la app o solicitud de persona, crea una solicitud de atención con prioridad.	El escalamiento crea una SolicitudAtencion en solicitudes_atencion, notifica al administrador y devuelve un radicado de 8 caracteres.	Alta	Parcialmente cumplido
RF-020	**Gestión de solicitudes de atención (administrador).** El administrador toma, responde y resuelve las solicitudes de atención.	Tomar cambia el estado a En Atención y registra al responsable; responder guarda la respuesta; resolver cambia a Resuelto con fecha de resolución.	Alta	Cumplido
RF-021	**Portal del cliente.** El cliente crea solicitudes de servicio o atención, consulta sus órdenes y ve las respuestas del equipo.	El cliente solo ve sus propias solicitudes; puede abrir el chat para preguntar sobre una orden.	Alta	Parcialmente cumplido

ID	Requisito y descripción	Criterio de aceptación	Prior.	Estado
RF-022	**Notificación de eventos a Make.** Al crear, actualizar o completar un servicio se envía un evento al webhook de Make; el administrador puede enviar un reporte manual y probar la conexión.	Crear servicio envía SERVICIO_CREADO; completarlo envía SERVICIO_COMPLETADO con el estado Completado.	Media	Cumplido
RF-023	**Restablecimiento de datos.** El administrador puede restablecer los datos de la aplicación.	La acción restablece los datos de la cuenta a su estado inicial.	Baja	No verificable

3.3 Requisitos no funcionales

ID	Categoría	Requisito	Criterio de aceptación	Estado
RNF-001	Seguridad	Las reglas de Firestore aplican mínimo privilegio: cada usuario accede a su árbol usuarios/{uid}; el rol no cambia tras el registro; cualquier otra ruta se deniega.	firestore.rules sin '|| true' ni comodines abiertos; desplegadas en servix-453fb.	Parcialmente cumplido
RNF-002	Seguridad	Ninguna clave de IA se empaqueta en el APK: el chat usa Firebase AI Logic.	No existe GEMINI_API_KEY en BuildConfig ni en el código.	Cumplido
RNF-003	Seguridad	Los atajos de acceso de prueba solo existen en builds debug.	El bloque de atajos está condicionado a BuildConfig.DEBUG.	Cumplido
RNF-004	Seguridad	El repositorio no contiene secretos: .env y local.properties están fuera de control de versiones y se documenta .env.example.	Sin credenciales en el repositorio.	Parcialmente cumplido
RNF-005	Compatibilidad	La app es compatible con Android 8.0 (API 26) o superior y apunta a la API 36.	minSdk 26 y targetSdk 36 en app/build.gradle.kts.	Cumplido
RNF-006	Mantenibilidad	Arquitectura en capas (presentación, dominio, datos, núcleo) con repositorios definidos por interfaz y DI manual en AppContainer.	Las capas dependen del dominio; los repositorios se sustituyen por dobles en pruebas.	Cumplido
RNF-007	Usabilidad	Interfaz Material 3 con identidad azul de ServiX, modo claro y oscuro y estados vacíos y de error comprensibles.	Paleta definida en core/theme/Color.kt; modo oscuro configurable.	Parcialmente cumplido
RNF-008	Fiabilidad	Si la IA falla, el sistema informa el error y no presenta respuestas predefinidas como si fueran de IA.	El caso de uso devuelve un mensaje de error y sugiere reintentar o pedir un asesor.	Parcialmente cumplido
RNF-009	Testabilidad	El proyecto cuenta con pruebas unitarias y de UI sobre JVM ejecutables con Gradle.	gradlew testDebugUnitTest finaliza sin fallos.	Cumplido
RNF-010	Portabilidad y CI	El proyecto compila con el wrapper de Gradle en un equipo limpio y en Azure Pipelines.	assembleDebug exitoso local y en el pipeline ServiX-CI.	Cumplido
RNF-011	Persistencia	Los datos se almacenan en Cloud Firestore con escuchas en tiempo real por usuario.	Los repositorios Firebase usan addSnapshotListener sobre usuarios/{uid}/...	Parcialmente cumplido
RNF-012	Mantenibilidad	Los componentes de presentación se concentran de forma razonable; el MainViewModel no debe crecer sin control.	MainViewModel.kt de unos 25 KB (candidato a dividirse).	Parcialmente cumplido

3.4 Modelo de datos lógico

Las entidades de dominio son Cliente, Servicio, SolicitudAtencion, Notificacion, Actividad, AuthUser, UserProfile y AppConfig. Se almacenan en Firestore como se muestra a continuación.

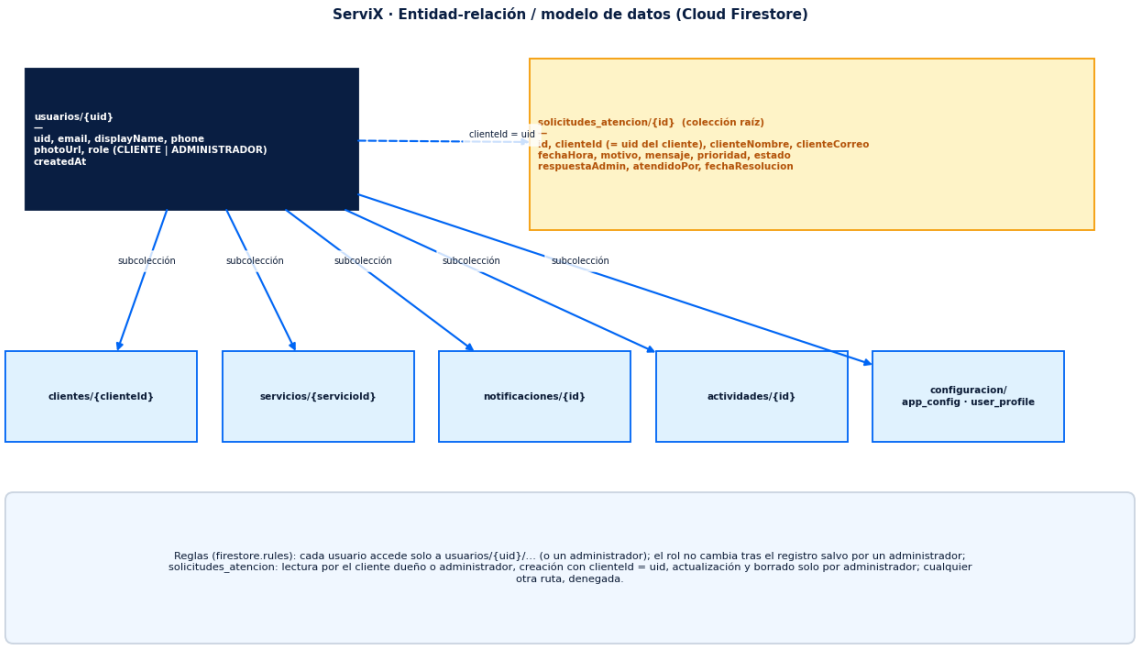


Figura 2. Modelo de datos en Cloud Firestore.

3.5 Restricciones de diseño y seguridad

- El rol se guarda en `usuarios/{uid}.role` y no puede cambiarse por el propio usuario.
- Las reglas de Firestore deniegan toda ruta no listada.
- No se incluyen claves de IA ni secretos en el APK ni en el repositorio.
- Riesgos residuales: el registro permite elegir el rol Administrador (H-15), el contenedor de dependencias degrada a repositorios simulados sin aviso (H-11) y la URL del webhook de Make está en el código (H-05).

4. Verificación y trazabilidad

4.1 Métodos de verificación

Método	Uso en ServiX
Prueba automática	29 pruebas unitarias y de UI con Robolectric ejecutadas con `gradlew testDebugUnitTest`.
Inspección	Revisión de código, reglas de Firestore y configuración.
Demostración	Pruebas manuales en dispositivo (pendientes).
Análisis estático	Android Lint (no ejecutado).

4.2 Trazabilidad requisito a casos de prueba

Requisito	Historia	Casos de prueba	Estado
RF-001	HU-001	CP-016, CP-033	Parcialmente cumplido
RF-002	HU-002	CP-014, CP-015, CP-030, CP-031, CP-032	Parcialmente cumplido
RF-003	HU-002	CP-017	Cumplido
RF-004	HU-003	CP-034	No verificable

Requisito	Historia	Casos de prueba	Estado
RF-005	HU-002	CP-003, CP-004, CP-025, CP-026, CP-027, CP-028, CP-029, CP-030, CP-031	Parcialmente cumplido
RF-006	HU-004	CP-005, CP-006, CP-007, CP-011, CP-012, CP-035	Cumplido
RF-007	HU-004	CP-009, CP-035	Cumplido
RF-008	HU-005	CP-035, CP-036	Parcialmente cumplido
RF-009	HU-004	CP-035	Parcialmente cumplido
RF-010	HU-006	CP-005, CP-008, CP-010, CP-012, CP-036	Cumplido
RF-011	HU-006	CP-008, CP-012, CP-023, CP-036	Cumplido
RF-012	HU-007	CP-028, CP-037	Parcialmente cumplido
RF-013	HU-007	CP-018, CP-037	Parcialmente cumplido
RF-014	HU-008	CP-037	No verificable
RF-015	HU-008	CP-025	Parcialmente cumplido
RF-016	HU-009	CP-029, CP-046	Parcialmente cumplido
RF-017	HU-009	CP-025	No verificable
RF-018	HU-010	CP-038, CP-048	No verificable
RF-019	HU-011	CP-018, CP-039	Parcialmente cumplido
RF-020	HU-012	CP-018, CP-019, CP-020, CP-021, CP-041	Cumplido
RF-021	HU-013	CP-040	Parcialmente cumplido
RF-022	HU-014	CP-022, CP-023, CP-024, CP-043	Cumplido
RF-023	HU-009	CP-029	No verificable

Los casos CP-001 a CP-029 son pruebas automáticas aprobadas; CP-030 a CP-048 figuran como **No ejecutado** hasta disponer de dispositivo o servicios reales; CP-049 y CP-050 son verificaciones de compilación y CI.

5. Mapeo a IEEE 830-1998

IEEE 830-1998	Sección de este documento
1. Introducción (propósito, alcance, definiciones, referencias, visión general)	1. Introducción
2. Descripción general (perspectiva, funciones, usuarios, restricciones, supuestos)	2. Descripción general
3.1 Interfaces externas	3.1
3.2 Requisitos funcionales	3.2
3.3 a 3.5 Rendimiento, restricciones y atributos (seguridad, mantenibilidad)	3.3 y 3.5
Apéndices	Matrices en `02_Matrices` y sección 4

6. Limitaciones de este documento

- Las funciones que dependen de Firebase real, Gemini y Make no se verificaron en dispositivo.
- No se midieron requisitos de rendimiento, por lo que no se declaran.
- Los requisitos se derivaron del código existente; el Product Owner debe validarlos con el cliente.